

Proposal: Initial Data for App Disk Deployments

Proposed by: Atlas (operations-manager) **Date:** 2026-03-28 **Revised:** 2026-05-31 (CEO review comments applied) **Status:** Proposed — awaiting Axle input on data storage location **Parent:** [2026-03-28-app-dev-agent.md](#) (Kit agent proposal)

Purpose

Some IDEA Apps need to start with initial data already present — curriculum content, presentations, reference files. The goal of this proposal is not to ask "where does data live on disk" but: **how can Kit create different named variants of an App** (e.g. a Kolibri for primary school vs a Kolibri for secondary school), each starting with different initial data, while keeping the App itself as data-agnostic as possible?

The App should be as data-less as possible. Data and App are separate concerns. The App defines how to run services; the data variant defines what content a school receives.

Terminology

- **App** — a [compose.yaml](#) plus associated config; defines how services run
 - **Data variant** — a named initial-data set for an App (e.g. [kolibri-primary](#), [kolibri-secondary](#)); produced by Kit; packaged onto a specific App Disk
 - **App Disk** — a physical USB disk containing one App plus one data variant
-

Three Data Loading Approaches

Approach 1 — Init-data mount (recommended default)

Kit starts an empty App instance and mounts an empty data directory into it. Kit then uses the running App's own interface (CLI, HTTP API, or admin tool) to download and organise content in an app-native way. Once the content is populated, Kit snapshots the data directory and packages it as a named data variant.

Kit flow:

1. Start App with `IDEA_STORE_DIR=temp`, mount empty `/data` volume
2. Use App's own tools to load content (e.g. Kolibri import channel X)
3. Stop App
4. Copy `/data` snapshot → `init-data/<variant-name>/`
5. Package into App Disk as `apps/<app>-<version>/init-data/<variant>/`

At first start on a real App Disk, the App detects an empty volume, copies from `init-data/<variant>/`, and starts normally. Subsequent starts skip the copy (volume already populated).

Why this approach is preferred: The data is loaded through the App's own mechanism — the same way a real teacher would load it. This means the data structure is always valid for that App version. Kit does not need to know the internal format.

Multiple variants: Kit can produce multiple `init-data` snapshots for the same App (e.g. `kolibri-primary`, `kolibri-secondary`) by repeating the process with different content. Each App Disk ships with exactly one variant; the variant is chosen at build time.

Example: Kolibri — Kit imports a primary-school channel via `kolibri manage importchannel`, snapshots the Kolibri data directory, and packages it as `init-data/kolibri-primary/`. A separate run imports a secondary-school channel and produces `init-data/kolibri-secondary/`.

Approach 2 — Content folder (compose-native reference)

Some Apps reference their content directly from the startup command or a mount defined in `compose.yaml` — the App reads from a well-known path on disk rather than from an internal volume. This is an architectural property of the App, not a size concern.

```
/disks/sda/  
  META.yaml  
  content/  
    wikipedia-en-2026.zim    ← Kiwix reads this at startup; no  
copy  
  apps/  
    kiwix-1.0/
```

```
compose.yaml          ← mounts content/ as read-only
volume
```

For these Apps, Kit places the data file(s) in the `content/` directory at disk build time. No copy-on-first-start step is needed — the App reads from `content/` directly.

Variants are achieved by packaging different content files for different disks.

Example: Kiwix — the ZIM file (Wikipedia, Khan Academy, etc.) is specified in the compose mount. Kit places the chosen ZIM file in `content/` at build time.

Approach 3 — Docker service manipulation

Kit adds a short-lived Docker service to `compose.yaml` that runs once on first start, directly manipulates the content inside the App container's volume (via SQL, filesystem writes, or the App's internal API), then exits.

Used when Kit knows exactly how the data must be represented inside the App — i.e. when the internal format is stable, documented, and Kit can write to it safely without going through the App's own interface.

```
# compose.yaml (simplified)
services:
  app:
    image: koenswings/nextcloud
    volumes:
      - nextcloud-data:/var/www/html/data

  init:
    image: koenswings/nextcloud-init
    volumes:
      - nextcloud-data:/var/www/html/data
      - ./init-data/presentations:/presentations:ro
    command: ["/init.sh"]          # copies presentations into
Nextcloud's data structure
    depends_on: [app]
    restart: "no"
```

Example: Nextcloud — Marco's presentations must appear in a specific Nextcloud folder structure. Kit uses an init service to copy them into the correct path inside the Nextcloud data volume on first start.

When to use: Only when Approach 1 is impractical (e.g. the App's own import tooling is too slow, requires a UI, or is not scriptable). Approach 3 creates a coupling between Kit and the App's internal data format — a risk if the App changes its storage layout.

Open Question: Where Does Initial Data Live?

This is the most significant unresolved question. For Approaches 1 and 3, Kit needs a source of initial data at build time. Options:

Option I — In the app repo (git)

Data lives in `init-data/<variant>/` directly in the app repo alongside `compose.yaml`.

Pro: Simple; data is versioned with the App; no external dependencies at build time. **Con:** Git is unsuitable for large binary files (Kolibri databases, ZIM files can be hundreds of MB to several GB). Bloats the repo and degrades git performance.

Verdict: Acceptable only for small, structured data (presentations, config files). Not viable for content-heavy Apps.

Option II — DockerHub (data-only images)

Kit builds a separate Docker image containing only data (**FROM scratch + COPY**). The image is pushed to DockerHub under `koenswings/`. At build time, Kit runs the image briefly to extract the data directory.

Pro: Versioned; reproducible; no git bloat; DockerHub pull works on Pi with internet. **Con:** DockerHub is a public image registry — not designed for large binary data. Large layers are slow to push/pull. Free tier has rate limits.

Verdict: Viable for medium-sized data sets. Worth considering for presentations and structured seed data. Not ideal for multi-GB content.

Option III — Dedicated local data disk (always-mounted)

A dedicated large-capacity USB disk is permanently mounted on the Pi at a known path (e.g. `/mnt/idea-data/`). Kit stores all source data here, organised by App and variant. At build time, Kit reads from this disk. It is never shipped to schools — it is a build input only.

Pro: No size limits; fast local access; no external dependencies; data can be updated independently of the App repo. **Con:** Requires a dedicated disk on the Pi; not part of the standard setup; needs a documented mount convention and backup strategy.

Verdict: The most practical option for large content (ZIM files, Kolibri channels). Should be the primary approach for content-heavy variants.

Option IV — Cloud storage (S3, R2, or similar)

Data is stored in cloud object storage. Kit downloads it at build time.

Pro: Unlimited size; accessible from any Pi; easy to update. **Con:** Requires internet access at build time; adds an external dependency; costs money for storage and egress; build fails if cloud is unreachable.

Verdict: Useful as a fallback or for distributing data to multiple Pi setups. Not suitable as the sole source — offline builds must remain possible.

Recommendation (pending Axle input)

App	Data approach	Data storage
Kolibri (primary/secondary)	Approach 1 (init-data mount)	Option III (local data disk)
Kiwix	Approach 2 (content folder)	Option III (local data disk)
Nextcloud (presentations)	Approach 3 (Docker init service)	Option I (in app repo — presentations are small)
Nextcloud (other data)	Approach 1	Option III
Seafile, Kolibri Studio	TBD at bootstrap	TBD

This recommendation is preliminary. Axle's input is needed on the local data disk approach — specifically: how does the Engine interact with permanently-mounted disks? Is there a risk the Engine tries to process the data disk as an App Disk? What mount path convention is safe?

A [\[From Atlas\] Opinion](#) cross-agent task has been created on Axle's board to solicit this input before the data storage approach is finalised.

Marco's presentations (Nextcloud)

Apply Approach 3 (Docker init service). Marco maintains a [presentations/](#) folder at [agents/agent-programme-manager/presentations/](#). Kit's Nextcloud init service copies from this path into the Nextcloud data volume at first start.

Marco is responsible for keeping this folder current. Kit detects changes (via a [From Marco] FYI cross-agent task) and rebuilds the Nextcloud init image.

Impact

- **Kit:** implements the appropriate data approach per App; owns init-data snapshots and init service images; manages the local data disk convention (if adopted)
- **Marco:** maintains `presentations/` folder; notifies Kit via MC task when updated
- **Axle:** input needed on local data disk safety (see open question above); engine must not try to process the data disk as an App Disk
- **Each app repo:** gains an `init-data/` directory or `content/` convention as appropriate